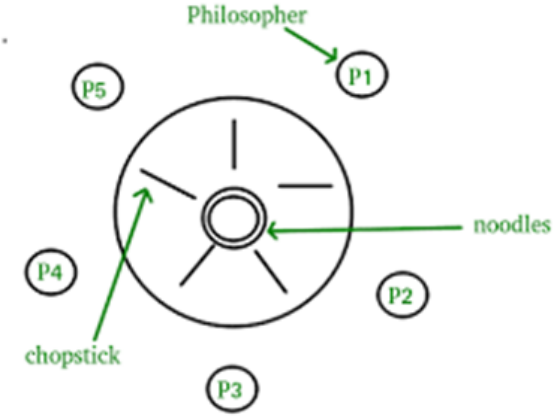
SE 375 SYSTEM PROGRAMMING SPRING 2025-2026
Laboratory Assignment 4
March 14, 2026

The Dining Philosophers Problem describes a scenario where five philosophers sit at a circular table. A single chopstick is placed between each pair of adjacent philosophers. To eat, a philosopher must acquire both the chopstick to their immediate left and the chopstick to their immediate right. However, because the chopsticks are shared resources, a conflict arises: if every philosopher simultaneously picks up the chopstick to their left, they will all be holding their single chopstick to the left while waiting indefinitely for the chopstick to their right to become available. This creates a state of deadlock, where no progress can be made because every participant is waiting for a resource held by another.



In this lab, you will implement a simplified version of the Dining Philosophers Problem in which there will be 3 philosophers and 3 chopsticks. Each philosopher and each chopstick will be represented as an object.

The purpose of the lab is to understand how deadlock occurs in concurrent programming and how a simple design change can prevent it. First, you will implement a version that may lead to deadlock. Then, you will implement a corrected version that avoids deadlock. You will only use the concepts seen during the lecture hours.

Part 1 – Deadlock Version

In this version, every philosopher first picks up the left chopstick and then tries to pick up the right chopstick. If the right one is not available, the philosopher waits until it becomes available. If all philosophers do this at the same time, each philosopher may hold one chopstick and wait forever for the other one. This is a deadlock.

A possible scenario that leads to a deadlock is as follows:

```
Philosopher 1 picks up Chopstick 1
Philosopher 2 picks up Chopstick 2
Philosopher 3 picks up Chopstick 3
Philosopher 1 waits for Chopstick 2
```

```
Philosopher 2 waits for Chopstick 3
Philosopher 3 waits for Chopstick 1
```

At this point, no philosopher can continue. Each one is waiting for a chopstick held by another philosopher. The program is stuck. Implement this scenario in Java.

Part 2 – Deadlock Solution

Solve the problem in Part 1 by applying certain procedures that prevent the system from entering a deadlock state. Modify the design of the program so that philosophers do not wait indefinitely for chopsticks.

In the original implementation, all philosophers attempt to pick up the left chopstick first and then the right chopstick. If all philosophers follow the same order simultaneously, a circular waiting condition may occur, leading to deadlock. To prevent this, a simple strategy can be applied. By applying this strategy, the circular waiting condition is broken.

You should modify the implementation accordingly and demonstrate that the program runs continuously without entering a deadlock state. Explain your solution on how it solves the deadlock problem by writing comments in the Java file.

You will submit 2 files: One with the deadlock problem in it. The other will be the solution.

Regulations

- You should use Java.
- Late submission is **not** allowed.
- This is an **individual** assignment. Plagiarism, copying, cheating, outsourcing the exam to another person or organization for with or without pay are considered as actions of academic dishonesty. Failure to maintain academic honesty may result in disciplinary action according to the Izmir University of Economics' disciplinary by law for students of institutions of higher education (https://www.ieu.edu.tr/en/bylaws/type/read/id/13).
- Submissions should be done to Blackboard. Submissions via e-mail will not be accepted.
- You should make sure to obey the specifications.

Submission

- You should submit java files to Blackboard.
- By submitting you pledge the following; "On my honor, as an Izmir University of Economics student, I affirm that I have not given or received any unauthorized help on this exam, and that this work is my own."